

BUILDING BACKEND SYSTEMS

Week Three

Making Databases Fast and Safe

Indexes, transactions, race conditions, ORMs, secrets management, and JWT authentication. The week where backend engineering becomes systems engineering.

WEEK Three of Six	DAYS 15 – 21	TOOLS PostgreSQL · Python · FastAPI
-----------------------------	------------------------	---

Contents

Day 15	Indexes and How Databases Find Data <i>B-Trees, EXPLAIN ANALYZE, sequential vs index scan, write cost</i>
Day 16	Transactions and ACID <i>Atomicity, consistency, isolation, durability, ROLLBACK</i>
Day 17	Concurrency: Race Conditions and Locks <i>Lost updates, SELECT FOR UPDATE, optimistic locking, deadlocks</i>
Day 18	ORMs: SQLAlchemy <i>Models, sessions, relationships, the N+1 problem, joinedload</i>
Day 19	Environment Variables and Secrets <i>.env, Pydantic Settings, secret rotation, production patterns</i>
Day 20	Authentication: Sessions and JWT <i>Password hashing, JWT structure, signing, token expiry</i>
Day 21 — Summary	Week Three Consolidation <i>The connecting thread, integration exercise, final commit</i>

WHAT THIS WEEK CHANGES

At the end of Week Two, you could write correct SQL. At the end of Week Three, you will understand how to make that SQL work correctly under conditions that Week Two never encountered: large data volumes, concurrent users, deployment environments, and authenticated requests.

Every concept this week is motivated by a specific failure mode. Indexes are motivated by slow queries. Transactions are motivated by partial writes. Locking is motivated by lost updates. Secrets management is motivated by data breaches. Authentication is motivated by unauthorised access. If you understand the failure mode, the solution will feel inevitable rather than arbitrary.

Before you begin Day 15

Ensure `university_db` exists and contains your students, courses, and enrollments from Week Two.

Create a folder: `mkdir week-03` in your repository.

Add `SECRET_KEY=your-long-random-string-here` to your `.env` file before Day 20.

Day 15

Indexes and How Databases Find Data

Week Three · Making Databases Fast and Safe

This is where backend engineering starts to become systems engineering. A query against five rows is never slow. A query against five hundred thousand rows can be instantaneous or take thirty seconds — and the difference is not the query, it is whether the database has been told where to look. Today you will see that difference with your own eyes.

You will create a table with half a million rows, run queries against it with and without indexes, measure the difference using `EXPLAIN ANALYZE`, and read the query plan output that the database produces. By the end of today, you will never create a production table without thinking about indexes first.

THEORY · 20 MINUTES

What a full table scan is

When you run `SELECT * FROM users WHERE email = 'x@example.com'`, the database has to find the row where the email matches. Without an index, there is only one way to do this: read every single row in the table from start to finish and check whether the email column matches. This is called a sequential scan or full table scan.

On a table with one hundred rows, a sequential scan is imperceptible. On a table with one million rows, the same query might take several seconds. On

a table with one hundred million rows, it might take minutes. The number of rows scanned grows linearly with the table size — and so does the query time.

What an index is

An index is a separate data structure that the database maintains alongside your table. It contains a sorted copy of one or more column values and pointers to the rows where each value lives. When you query with a WHERE clause on an indexed column, the database uses the index to jump directly to the matching rows rather than scanning the entire table.

The most common index type in PostgreSQL is the B-Tree index. A B-Tree is a balanced tree structure where each node contains sorted values and pointers to child nodes. To find a value, the database starts at the root, compares the target value with the values in the node, and follows the appropriate branch — either left (smaller), right (larger), or direct (match). Because the tree is balanced, this takes $O(\log n)$ comparisons regardless of table size. For one million rows, that is about twenty comparisons instead of one million.

The cost of an index

Indexes are not free. Every index you create must be maintained whenever data in the indexed column changes. An INSERT adds an entry to every index on the table. An UPDATE on an indexed column removes the old entry and adds a new one. A DELETE removes the entry. On a table with many indexes, write operations slow down noticeably. The tradeoff is read speed versus write speed.

The practical rule: add an index to any column you regularly query with a WHERE clause, ORDER BY, or JOIN condition. Do not add indexes speculatively. Index the columns your queries actually use, measure the impact, and only keep indexes that demonstrably improve performance.

EXPLAIN ANALYZE — reading query plans

EXPLAIN ANALYZE runs a query and shows you how the database executed it — which indexes it used, how many rows it scanned, and how long each step

took. Learning to read this output is one of the most practically valuable skills in backend engineering.

Term	Meaning
<i>Seq Scan</i>	Sequential scan — the database read every row in the table. Slow on large tables.
<i>Index Scan</i>	The database used an index to find specific rows. Fast for selective queries.
<i>Index Only Scan</i>	The database answered the query entirely from the index without touching the table. The fastest.
<i>Bitmap Heap Scan</i>	The database used an index to build a bitmap of matching pages, then fetched those pages. Efficient for queries matching many rows.
<i>rows=N</i>	The planner's estimate of how many rows this step will return.
<i>actual rows=N</i>	How many rows the step actually returned. A large gap between estimate and actual signals stale statistics.
<i>cost=N..M</i>	The planner's estimated cost. The second number is the total cost to return all rows.
<i>actual time=N..M</i>	The actual time in milliseconds for this step.

PRACTICAL · 60 MINUTES

This practical requires generating a large table. The Faker library makes this straightforward. Install it first.

```
pip install faker psycpg2-binary
```

Install dependencies before starting

1. Generate 500,000 rows (15 minutes)

Create a Python script that populates a `large_users` table. Use batch inserts — inserting one row at a time for half a million rows is extremely slow.

```

# week-03/day15_seed.py
import psycopg2
from faker import Faker
import time

fake = Faker()
conn = psycopg2.connect('dbname=university_db user=postgres
password=YOUR_PASSWORD')
cur = conn.cursor()

# Create the table
cur.execute('''
    DROP TABLE IF EXISTS large_users;
    CREATE TABLE large_users (
        id            INTEGER GENERATED ALWAYS AS IDENTITY
PRIMARY KEY,
        email         TEXT NOT NULL,
        name          TEXT NOT NULL,
        country       TEXT NOT NULL,
        created_at    TIMESTAMPTZ NOT NULL DEFAULT NOW()
    );
''')
conn.commit()

# Insert in batches of 10,000 for speed
TOTAL = 500_000
BATCH = 10_000
start = time.time()

for batch_start in range(0, TOTAL, BATCH):
    rows = [(fake.unique.email(), fake.name(),
fake.country_code())
            for _ in range(BATCH)]
    cur.executemany(
        'INSERT INTO large_users (email, name, country)
VALUES (%s, %s, %s)',
        rows
    )
    conn.commit()
    print(f'Inserted {batch_start + BATCH:,} rows...')

print(f'Done in {time.time() - start:.1f}s')
cur.close(); conn.close()

```

week-03/day15_seed.py — this will take 1–3 minutes to run

2. Measure query time without an index (15 minutes)

Run these queries in psql. Read every line of the EXPLAIN ANALYZE output before continuing.

```
-- First, confirm row count
SELECT COUNT(*) FROM large_users;

-- Query without index — watch for 'Seq Scan'
EXPLAIN ANALYZE
SELECT * FROM large_users WHERE email = 'test@example.com';

-- Run a real lookup using an email you know exists
-- First find one:
SELECT email FROM large_users LIMIT 1;

-- Now measure the lookup:
EXPLAIN ANALYZE
SELECT * FROM large_users WHERE email = '<paste email here>';

-- Write down the actual time= value from the output
```

Note the 'Seq Scan' line and the actual time — you will compare this after adding the index

3. Add the index and measure again (15 minutes)

Create the index and run the exact same queries. Compare the query plan and the execution time.

```
-- Create a B-Tree index on the email column
CREATE INDEX idx_large_users_email ON large_users(email);

-- Run the same EXPLAIN ANALYZE — now look for 'Index Scan'
EXPLAIN ANALYZE
SELECT * FROM large_users WHERE email = '<same email>';

-- Try an ORDER BY query without an index on country
EXPLAIN ANALYZE
SELECT name, country FROM large_users ORDER BY country LIMIT 20;

-- Create an index on country and repeat
CREATE INDEX idx_large_users_country ON large_users(country);

EXPLAIN ANALYZE
SELECT name, country FROM large_users ORDER BY country LIMIT 20;
```

Record both actual time values — the ratio is the story

4. Test the cost of indexes on writes (15 minutes)

Measure how the indexes affect INSERT speed. This completes the picture.

```
-- Time an INSERT without extra indexes on a clean table
-- (observe: the table already has 2 indexes — email and
country)
\timing on

INSERT INTO large_users (email, name, country)
SELECT 'bulk_' || g || '@test.com', 'Test User', 'GH'
FROM generate_series(1, 10000) g;

-- Drop the indexes and run the same INSERT again
DROP INDEX idx_large_users_email;
DROP INDEX idx_large_users_country;

INSERT INTO large_users (email, name, country)
SELECT 'bulk2_' || g || '@test.com', 'Test User', 'GH'
FROM generate_series(1, 10000) g;

\timing off
-- Compare the two INSERT times. The difference is index
maintenance cost.
```

Re-create the indexes after this test — you will use `large_users` in later exercises

REFLECTION · 10 MINUTES

These questions go beyond syntax to the engineering decisions that indexes force.

Question 1

You measured the time difference between a sequential scan and an index scan on 500,000 rows. Extrapolate: if a production table has 50 million rows, what would you expect the ratio to be? What changes at that scale that does not change at 500,000?

Question 2

You created two indexes — one on email and one on country. If you also frequently query `WHERE email = x AND country = y`, would those two separate indexes be sufficient, or would you want a composite index on (email, country)? What is the difference?

Question 3

An INSERT on a table with two indexes is slower than an INSERT on the same table with no indexes. Given this, should you remove indexes from a production table before a large data migration? What are the tradeoffs?

Question 4

EXPLAIN ANALYZE showed an actual rows estimate from the planner. If the estimate is wildly different from the actual rows, what does that tell you about the database's statistics, and what command would you run to fix it?

BEFORE TOMORROW

Today you understood why some queries are fast and others are slow. Tomorrow you learn why some database operations need to be atomic — what happens when a bank transfer updates two rows and the server crashes between the first update and the second. The answer is transactions, and the concept is one of the most important in all of backend engineering.

Commit before you close your laptop

```
Save week-03/day15_seed.py and week-03/day15_indexes.sql
git commit -m "Day 15: indexes, B-Tree, EXPLAIN ANALYZE, write cost measurement"
```


Day 16

Transactions and ACID

Week Three · Making Databases Fast and Safe

Imagine a bank transfer: you deduct one hundred dollars from account A and add one hundred dollars to account B. Two UPDATE statements. What happens if the database processes the first UPDATE, the server crashes, and the second UPDATE never runs? Account A has lost one hundred dollars. Account B has gained nothing. The money has disappeared.

This problem is exactly what transactions solve. A transaction is a unit of work that either completely succeeds or completely fails. No partial results. No half-finished operations. Today you will build the bank transfer problem, crash it deliberately, and then fix it with transactions. The concept will stay with you permanently because you will have felt the problem before you learned the solution.

THEORY · 20 MINUTES

What a transaction is

A transaction is a sequence of database operations that the database treats as a single unit. The transaction begins with BEGIN, ends with COMMIT if everything succeeded, or ends with ROLLBACK if something went wrong. Between BEGIN and COMMIT, every change you make is invisible to other sessions — they see only the state from before your transaction began.

In PostgreSQL, every individual SQL statement is automatically wrapped in its own transaction if you do not explicitly start one. When you run a standalone INSERT, the database begins a transaction, executes the INSERT, and immediately commits. Understanding this is important — it means the concept of a transaction is always present, even when you do not write BEGIN explicitly.

ACID — the four guarantees

ACID is an acronym for the four properties that a database transaction must provide. These are guarantees, not suggestions — the database engine enforces them.

Term	Meaning
<i>Atomicity</i>	All operations in the transaction succeed, or none of them do. There is no partial commit. If the server crashes mid-transaction, the database rolls back to the state before the transaction began.
<i>Consistency</i>	A transaction brings the database from one valid state to another valid state. All constraints — NOT NULL, UNIQUE, foreign keys, CHECK — are enforced at commit time. A transaction that would violate a constraint is rolled back.
<i>Isolation</i>	Concurrent transactions behave as if they were running sequentially. Changes made by one transaction are not visible to others until that transaction commits.
<i>Durability</i>	Once a transaction commits, it is permanent. Even if the server crashes immediately after the commit, the changes survive because they have been written to durable storage.

Isolation levels

The I in ACID — isolation — has degrees. PostgreSQL supports four isolation levels, each offering a different tradeoff between consistency and concurrency.

Term	Meaning
------	---------

<i>Read Uncommitted</i>	Not supported in PostgreSQL — treated as Read Committed.
<i>Read Committed</i>	Default. A query sees only rows committed before the query began. Two queries within the same transaction can see different data if another transaction commits between them.
<i>Repeatable Read</i>	A transaction sees a consistent snapshot from the moment it began. Two queries within the same transaction will always see the same data, even if another transaction commits between them.
<i>Serializable</i>	The strictest level. Transactions behave as if they ran one at a time, in serial order. PostgreSQL detects conflicts and rolls back transactions that would have produced results inconsistent with serial execution.

ROLLBACK and SAVEPOINT

ROLLBACK undoes all changes made since the last BEGIN. If you are inside a transaction and something goes wrong — a constraint violation, an application error, a network failure — you roll back and the database is left in exactly the state it was before you began.

SAVEPOINT allows partial rollbacks within a long transaction. You set a savepoint with SAVEPOINT my_point, and if a later part of the transaction fails, you can roll back only to SAVEPOINT my_point rather than rolling back the entire transaction. This is useful in complex operations where some steps succeeded and should be preserved.

PRACTICAL · 60 MINUTES

Today's practical has three distinct phases: break the bank transfer without transactions, fix it with transactions, and then test edge cases. Work through each phase completely before moving to the next.

1. Set up the bank accounts table (5 minutes)

Create the schema and seed some accounts.

```
-- In psql or save as week-03/day16_transactions.sql
CREATE TABLE accounts (
    id          INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    owner       TEXT          NOT NULL,
    balance     NUMERIC(12,2) NOT NULL CHECK (balance >= 0)
);

INSERT INTO accounts (owner, balance) VALUES
    ('Amara', 1000.00),
    ('Kwame', 500.00),
    ('Pivota', 0.00); -- the business account
```

The CHECK (balance >= 0) constraint prevents negative balances at the database level

2. Break it without a transaction (15 minutes)

Simulate a server crash between the two UPDATE statements and observe the inconsistent state.

```
-- Step 1: Check initial balances
SELECT * FROM accounts;

-- Step 2: Transfer 200 from Amara to Kwame — but 'crash'
-- after the first UPDATE
UPDATE accounts SET balance = balance - 200 WHERE owner =
'Amara';

-- Simulate a crash here: close psql without running the
-- second UPDATE
-- \q

-- Reconnect and check balances
SELECT * FROM accounts;
-- Amara is down 200. Kwame has not received anything. Money
-- is gone.

-- Reset for the next test
UPDATE accounts SET balance = 1000 WHERE owner = 'Amara';
```

This broken state is why every bank transfer in history uses a transaction

3. Fix it with a transaction — Python implementation (25 minutes)

Build a Python function that performs a transfer atomically. This is the pattern you will use in real applications.


```

# week-03/day16_transfer.py
import psycopg2
from decimal import Decimal

DB_URL = 'dbname=university_db user=postgres
password=YOUR_PASSWORD'

def transfer(from_owner: str, to_owner: str, amount: Decimal)
-> dict:
    """Transfer amount from one account to another
    atomically."""
    if amount <= 0:
        raise ValueError('Transfer amount must be positive')

    conn = psycopg2.connect(DB_URL)
    try:
        with conn:
            # psycopg2's context manager auto-commits on
            success,
            # auto-rolls back on exception
            with conn.cursor() as cur:
                # Deduct from sender - will raise if balance
                < amount (CHECK constraint)
                cur.execute(
                    'UPDATE accounts SET balance = balance -
                    %s WHERE owner = %s'
                    ' RETURNING balance',
                    (amount, from_owner)
                )
                row = cur.fetchone()
                if row is None:
                    raise ValueError(f'Account not found:
                    {from_owner}')

                # Credit recipient
                cur.execute(
                    'UPDATE accounts SET balance = balance +
                    %s WHERE owner = %s'
                    ' RETURNING balance',
                    (amount, to_owner)
                )
                row = cur.fetchone()
                if row is None:
                    raise ValueError(f'Account not found:
                    {to_owner}')

```

```

        return {'status': 'ok', 'amount':
str(amount)}

    except Exception as e:
        # Transaction was rolled back automatically by the
context manager
        return {'status': 'error', 'reason': str(e)}
    finally:
        conn.close()

# Test it
print(transfer('Amara', 'Kwame', Decimal('150')))
print(transfer('Kwame', 'Amara', Decimal('9999'))) # Should
fail - insufficient funds
print(transfer('Amara', 'Nobody', Decimal('10'))) # Should
fail - recipient not found

```

week-03/day16_transfer.py — verify all three tests and check account balances in psql after each run

4. Test the CHECK constraint as a safety net (15 minutes)

Try to create a transaction that violates the CHECK constraint and observe the automatic rollback.

```

-- In psql - try to overdraw an account within a transaction
BEGIN;
UPDATE accounts SET balance = balance - 50000 WHERE owner =
'Amara';
-- The UPDATE fails with: ERROR: new row violates check
constraint
-- The transaction is now in an aborted state
SELECT * FROM accounts; -- try this - observe the error
ROLLBACK; -- now the transaction is closed
SELECT * FROM accounts; -- balances unchanged

```

In psql, any command after a failed statement in an open transaction returns 'ERROR: current transaction is aborted' — you must ROLLBACK before continuing

REFLECTION · 10 MINUTES

Answer these questions to consolidate the concept.

Question 1

The transfer function uses psycopg2's context manager which auto-commits on success and auto-rolls back on exception. What would happen if you forgot to use the context manager and an exception occurred after the first UPDATE but before the second?

Question 2

You set a CHECK (balance >= 0) constraint on the accounts table. The application code also validates that the amount is positive. These are two layers of protection for the same invariant. Which layer is more important and why? Could you remove one of them safely?

Question 3

Describe in plain English what Atomicity, Consistency, Isolation, and Durability each mean using the bank transfer scenario — not the abstract definitions.

Question 4

The default isolation level in PostgreSQL is Read Committed. For the bank transfer use case, is Read Committed sufficient or would you need a stricter isolation level? What scenario could go wrong at Read Committed that Repeatable Read would prevent?

BEFORE TOMORROW

Tomorrow is the most surprising day of Week Three. You will write a simple counter endpoint, hammer it with concurrent requests, and watch it produce the wrong answer — even though each individual operation looks correct. This is the concurrency problem, and it is the most common source of silent data corruption in production backend systems.

Commit before you close your laptop

Save week-03/day16_transactions.sql and week-03/day16_transfer.py

```
git commit -m "Day 16: transactions, ACID, bank transfer, CHECK  
constraints"
```


Day 17

Concurrency: Race Conditions and Locks

Week Three · Making Databases Fast and Safe

A counter that increments by one for every request sounds trivial. Read the current value, add one, write it back. Three operations, completely correct logic. And yet if you send one hundred requests simultaneously, the final count will almost certainly be less than one hundred. This is a race condition, and it is one of the most common sources of silent data corruption in production systems.

Today you will create this problem deliberately, observe it, understand exactly why it happens, and learn the tools that fix it. By the end of this day you will never write a read-modify-write operation without thinking about concurrent access.

THEORY · 20 MINUTES

What a race condition is

A race condition occurs when two or more operations read the same data, make a decision based on that data, and then write back — and the writes conflict because both reads happened before either write completed. The read-modify-write sequence that looks atomic in application code is not atomic at the database level unless you explicitly make it so.

```

-- What your code does (one request)
1. SELECT value FROM counters WHERE id = 1;    -- returns 42
2. new_value = 42 + 1 = 43
3. UPDATE counters SET value = 43 WHERE id = 1;

-- What happens with 100 concurrent requests
Request 1: reads 42, computes 43
Request 2: reads 42, computes 43    <-- before request 1
writes
Request 3: reads 42, computes 43    <-- before either writes
...
All 100 requests read 42, all write 43.
Final value: 43.  Should be: 142.

```

This is the lost update problem — 99 increments were silently discarded

Locks — how the database serialises access

A lock is a mechanism that prevents two operations from accessing the same data simultaneously. The database automatically acquires locks on rows it modifies during a transaction. These are called row-level locks. A transaction that has locked a row prevents other transactions from modifying (but not reading) that row until the lock is released at commit or rollback.

`SELECT FOR UPDATE` is the explicit version. It reads a row and immediately locks it, preventing any other transaction from modifying that row until your transaction completes. This is pessimistic locking — you assume conflict will occur and prevent it in advance. The alternative is optimistic locking, where you allow concurrent reads and detect conflicts at write time.

Deadlocks

A deadlock occurs when two transactions each hold a lock the other needs. Transaction A holds a lock on row 1 and wants row 2. Transaction B holds a lock on row 2 and wants row 1. Both are waiting for the other. Neither can proceed. PostgreSQL detects this situation and automatically rolls back one of the transactions with an error. The rolled-back transaction can be retried.

The most reliable way to prevent deadlocks is to always acquire locks in the same order. If every transaction that touches rows 1 and 2 always locks row 1 first, then row 2, deadlocks are impossible because no circular dependency can form.

Atomic SQL updates — the correct solution

For simple increment operations, the best solution is not locking — it is writing an atomic SQL statement that does the read-modify-write entirely inside the database engine, where it cannot be interrupted by another connection.

```
-- NOT atomic: read in Python, write in SQL
value = cur.execute('SELECT value FROM counters WHERE id = 1')
cur.execute('UPDATE counters SET value = %s WHERE id = 1',
            (value + 1,))

-- ATOMIC: the entire operation happens inside the database
UPDATE counters SET value = value + 1 WHERE id = 1;
```

The atomic version cannot be interrupted — the database increments the value as a single indivisible operation

PRACTICAL · 60 MINUTES

This practical requires a running FastAPI server and a tool to send concurrent requests. Install httpx for the concurrent client.

```
pip install httpx fastapi uvicorn psycpg2-binary
```

Install dependencies

1. Create the counter table and a broken endpoint (15 minutes)

Build the endpoint with the race condition deliberately present.

```
-- In psql: create the counter
CREATE TABLE counters (
    id      INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
```



```

        name TEXT          NOT NULL UNIQUE,
        value INTEGER       NOT NULL DEFAULT 0
    );
    INSERT INTO counters (name) VALUES ('page_views');

```

week-03/day17_setup.sql

```

# week-03/day17_broken_counter.py
import psycopg2
from fastapi import FastAPI

app = FastAPI()
DB = 'dbname=university_db user=postgres
password=YOUR_PASSWORD'

@app.post('/increment')
def increment():
    conn = psycopg2.connect(DB)
    cur = conn.cursor()

    # BROKEN: read-modify-write is not atomic
    cur.execute('SELECT value FROM counters WHERE name = %s',
                ('page_views',))
    value = cur.fetchone()[0]
    new_value = value + 1
    cur.execute('UPDATE counters SET value = %s WHERE name =
                %s', (new_value, 'page_views'))
    conn.commit()
    conn.close()
    return {'value': new_value}

@app.get('/value')
def get_value():
    conn = psycopg2.connect(DB)
    cur = conn.cursor()
    cur.execute('SELECT value FROM counters WHERE name = %s',
                ('page_views',))
    value = cur.fetchone()[0]
    conn.close()
    return {'value': value}

```

week-03/day17_broken_counter.py — run with: `uvicorn day17_broken_counter:app --reload`

2. Hammer it with concurrent requests and observe the race (15 minutes)

Send 100 requests simultaneously and compare the final count with the expected value.

```
# week-03/day17_hammer.py
import asyncio, httpx

async def hammer(n: int = 100):
    async with httpx.AsyncClient() as client:
        tasks =
        [client.post('http://localhost:8000/increment') for _ in
         range(n)]
        await asyncio.gather(*tasks)

    # Check the final value
    async with httpx.AsyncClient() as client:
        r = await client.get('http://localhost:8000/value')
        print(f'Expected: {n}, Got: {r.json()["value"]}')

asyncio.run(hammer(100))
```

week-03/day17_hammer.py — reset the counter to 0 in psql before each test run

3. Fix it three ways and compare (30 minutes)

Implement three different fixes and test each one with the hammer. Understand the tradeoffs between them.

```
# week-03/day17_fixed_counter.py — three endpoints, each
using a different fix
from fastapi import FastAPI
import psycopg2

app = FastAPI()
DB = 'dbname=university db user=postgres
password=YOUR_PASSWORD'

# FIX 1: Atomic SQL — the simplest and fastest solution
@app.post('/increment/atomic')
def increment_atomic():
    conn = psycopg2.connect(DB)
    cur = conn.cursor()
    cur.execute(
        'UPDATE counters SET value = value + 1 WHERE name =
%s RETURNING value',
        ('page_views',)
    )
```

```

        value = cur.fetchone()[0]
        conn.commit(); conn.close()
        return {'value': value, 'method': 'atomic'}

# FIX 2: SELECT FOR UPDATE - pessimistic locking
# Use when you need to read the value before deciding what to
# write
@app.post('/increment/lock')
def increment_lock():
    conn = psycopg2.connect(DB)
    cur = conn.cursor()
    cur.execute('BEGIN')
    # FOR UPDATE locks the row - other transactions wait
    cur.execute('SELECT value FROM counters WHERE name = %s
FOR UPDATE', ('page_views',))
    value = cur.fetchone()[0]
    cur.execute('UPDATE counters SET value = %s WHERE name =
%s', (value + 1, 'page_views'))
    conn.commit(); conn.close()
    return {'value': value + 1, 'method':
'select_for_update'}

# FIX 3: Optimistic locking - retry on conflict
# Use when contention is rare and reads are frequent
@app.post('/increment/optimistic')
def increment_optimistic():
    for _ in range(5): # retry up to 5 times
        conn = psycopg2.connect(DB)
        cur = conn.cursor()
        cur.execute('SELECT value FROM counters WHERE name =
%s', ('page_views',))
        value = cur.fetchone()[0]
        # Only update if value hasn't changed since we read
        it
        cur.execute(
            'UPDATE counters SET value = %s WHERE name = %s
AND value = %s RETURNING value',
            (value + 1, 'page_views', value)
        )
        updated = cur.fetchone()
        conn.commit(); conn.close()
        if updated:
            return {'value': updated[0], 'method':
'optimistic'}
    return {'error': 'max retries exceeded'}

```

week-03/day17_fixed_counter.py — test each endpoint with the hammer script, resetting the counter before each test

REFLECTION · 10 MINUTES

These questions ask you to reason about tradeoffs rather than recall facts.

Question 1

You implemented three fixes for the race condition. For a simple counter, the atomic SQL fix is clearly best. For what kind of operation would you need SELECT FOR UPDATE instead? Give a concrete example from a real application.

Question 2

Optimistic locking retries on conflict. What happens if the contention is so high that the retry limit is always exceeded? What does this mean for a production endpoint that uses optimistic locking?

Question 3

Describe a scenario where a deadlock could occur in the bank transfer from Day 16. Two accounts, two simultaneous transfers going in opposite directions. Trace through the sequence of lock acquisitions and explain where the deadlock forms.

Question 4

SELECT FOR UPDATE serialises access to a row — requests queue up and execute one at a time. For a popular counter endpoint receiving thousands of requests per second, what is the performance implication of this serialisation? What alternative architecture would you use instead?

BEFORE TOMORROW

Tomorrow you meet SQLAlchemy — the ORM that translates between Python objects and SQL. After three days of writing raw SQL, you will appreciate exactly what the ORM is doing for you and exactly what it is hiding. You will also encounter the N+1 problem: a class of performance bug that ORMs make easy to create and hard to notice.

Commit before you close your laptop

Save all three files in week-03/

```
git commit -m "Day 17: race conditions, locks, SELECT FOR UPDATE,  
optimistic locking"
```


Day 18

ORMs: SQLAlchemy

Week Three · Making Databases Fast and Safe

An ORM — Object-Relational Mapper — lets you interact with your database using Python objects rather than raw SQL strings. Instead of writing `SELECT * FROM students WHERE id = 1`, you write `Student.query.get(1)`. The ORM generates the SQL for you, maps the result to a Python object, and handles the connection lifecycle.

After three days of raw SQL, you now understand exactly what the ORM is doing underneath. That understanding is what allows you to use an ORM effectively — knowing when to trust it, when to bypass it, and how to catch the performance problems it can silently introduce. The most important of those problems is the N+1 query, and you will create it and fix it today.

THEORY · 20 MINUTES

What an ORM gives you

An ORM provides three things: a way to define your database schema as Python classes (models), a query interface that translates Python method calls into SQL, and a session that manages the connection and tracks changes to objects so it knows what to write to the database when you commit.

The benefits are real. You write less SQL by hand. Your models are Python objects with methods, which makes your application code more

expressive. Schema migrations become manageable. Type checking works across your data layer. For the vast majority of application code — CRUD operations, filtered lookups, simple joins — the ORM produces correct and reasonably efficient SQL.

What an ORM hides — and why that matters

The ORM generates SQL automatically, which means it can generate bad SQL automatically. The most common ORM-generated performance problem is the N+1 query: a query that fetches N parent objects, then issues a separate query for each one to fetch its related objects — N+1 queries total instead of one JOIN query.

```
# The N+1 problem — looks innocent, executes 101 queries

students = session.query(Student).all()      # Query 1: fetch all
students                                     #
for student in students:                     # 100 iterations
    print(student.enrollments)               # Query 2-101: one per
student                                     #

# What actually happens in SQL:
# SELECT * FROM students;
# SELECT * FROM enrollments WHERE student_id = 1;
# SELECT * FROM enrollments WHERE student_id = 2;
# ... 98 more queries

# The fix — eager loading with joinedload:
from sqlalchemy.orm import joinedload
students =
session.query(Student).options(joinedload(Student.enrollments)).all()
# Now it's ONE query with a JOIN
```

The N+1 problem is silent — your code produces correct results, just slowly

SQLAlchemy core concepts

Term	Meaning
<i>Engine</i>	The connection factory. Created once at application startup, reused across requests.

<i>Session</i>	A unit of work. Tracks changes to objects and coordinates with the database.
<i>Model / Base</i>	A Python class that maps to a database table. Inherits from a declarative Base.
<i>relationship()</i>	Defines a relationship between two models. SQLAlchemy uses it to generate JOIN queries.
<i>lazy loading</i>	The default — related objects are fetched only when you access them. Causes N+1 if you loop.
<i>eager loading</i>	Related objects are fetched in the same query using <code>joinedload()</code> or <code>subqueryload()</code> .
<i>echo=True</i>	A debug flag on the Engine that prints every SQL statement to the console. Essential for understanding what the ORM is doing.

PRACTICAL · 60 MINUTES

Today you rebuild the Student Management API from Day 6 using SQLAlchemy with a real PostgreSQL database. Then you create the N+1 problem deliberately and fix it.

1. Set up SQLAlchemy (10 minutes)

Install the dependencies and create the database configuration module.

```
pip install sqlalchemy psycopg2-binary
```

Install dependencies

```
# week-03/day18_database.py
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker, DeclarativeBase

DATABASE_URL =
'postgresql://postgres:YOUR_PASSWORD@localhost/university_db'

# echo=True prints all SQL — ALWAYS enable this during
development
engine = create_engine(DATABASE_URL, echo=True)
```

```

SessionLocal = sessionmaker(autocommit=False,
                             autoflush=False, bind=engine)

class Base(DeclarativeBase):
    pass

def get_db():
    """FastAPI dependency - yields a db session and ensures
    it is closed."""
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

```

week-03/day18_database.py

2. Define models (15 minutes)

Write the SQLAlchemy models for students, courses, and enrollments. Read every line — notice how the `relationship()` calls mirror the foreign keys in your SQL schema.

```

# week-03/day18_models.py
from sqlalchemy import Integer, String, Numeric, DateTime,
ForeignKey, func
from sqlalchemy.orm import mapped_column, Mapped,
relationship
from datetime import datetime
from .day18_database import Base

class Student(Base):
    __tablename__ = 'students'

    id: Mapped[int] = mapped_column(Integer,
primary_key=True)
    name: Mapped[str] = mapped_column(String,
nullable=False)
    email: Mapped[str] = mapped_column(String,
nullable=False, unique=True)
    year: Mapped[int] = mapped_column(Integer,
nullable=False)
    enrolled_at: Mapped[datetime] =
mapped_column(DateTime(timezone=True),
server_default=func.now())

```

```

    # Relationship — lazy loaded by default
    enrollments: Mapped[list['Enrollment']] =
relationship('Enrollment', back_populates='student')

class Course(Base):
    __tablename__ = 'courses'

    id: Mapped[int] = mapped_column(Integer,
primary_key=True)
    title: Mapped[str] = mapped_column(String,
nullable=False)
    code: Mapped[str] = mapped_column(String,
nullable=False, unique=True)
    credits: Mapped[int] = mapped_column(Integer,
nullable=False)

    enrollments: Mapped[list['Enrollment']] =
relationship('Enrollment', back_populates='course')

class Enrollment(Base):
    __tablename__ = 'enrollments'

    id: Mapped[int] = mapped_column(Integer,
primary_key=True)
    student_id: Mapped[int] =
mapped_column(ForeignKey('students.id'), nullable=False)
    course_id: Mapped[int] =
mapped_column(ForeignKey('courses.id'), nullable=False)

    student: Mapped['Student'] = relationship('Student',
back_populates='enrollments')
    course: Mapped['Course'] = relationship('Course',
back_populates='enrollments')

```

week-03/day18_models.py — notice how relationship() mirrors the REFERENCES in your SQL schema

3. Create and fix the N+1 problem (35 minutes)

First trigger the N+1 problem, count the queries in the console output, then fix it with joinedload and compare.

```

# week-03/day18 n1 demo.py
from sqlalchemy.orm import joinedload
from .day18_database import SessionLocal
from .day18 models import Student

```

```

db = SessionLocal()

print('\n--- N+1 PROBLEM (watch the SQL output) ---')
students = db.query(Student).all()
for s in students:
    # This triggers a new SELECT query for each student
    print(f'{s.name}: {len(s.enrollments)} enrollments')

print(f'\nTotal queries: {len(students) + 1}')
print('(1 for students + 1 per student for enrollments)')

print('\n--- FIXED WITH joinedload (count the SQL statements now) ---')
students =
db.query(Student).options(joinedload(Student.enrollments)).all()
for s in students:
    print(f'{s.name}: {len(s.enrollments)} enrollments')

print('Total queries: 1 (one JOIN query fetched everything)')

db.close()

```

Run this and count the SQL statements printed in the console — the difference is stark

REFLECTION · 10 MINUTES

These questions connect ORM behaviour to real application performance.

Question 1

The N+1 problem produces correct results — just slowly. A table with 5 students and 20 total enrollments generates 6 queries. The same code on a table with 10,000 students generates 10,001 queries. At what point does this become a production incident?

Question 2

You set `echo=True` on the engine during development. What are the risks of leaving `echo=True` in production? What should you use instead to monitor query performance in a production application?

Question 3

The ORM generated a JOIN query when you used `joinedload`. Run the same query with `echo=True` and read the generated SQL. Is it the same JOIN you would have written by hand? What differences do you notice?

Question 4

When would you bypass the ORM entirely and use raw SQL? Name three specific scenarios from a real application where raw SQL is the better choice.

BEFORE TOMORROW

Tomorrow you tackle a topic that causes more production incidents than almost anything else: secrets management. Hardcoded database passwords, committed API keys, leaked environment variables — these are not theoretical threats. They are the reason for real security breaches at real companies. Tomorrow you clean up your own code first.

Commit before you close your laptop

Save all `week-03/day18_*.py` files

```
git commit -m "Day 18: SQLAlchemy ORM, models, N+1 problem,
joinedload fix"
```


Day 19

Environment Variables and Secrets

Week Three · Making Databases Fast and Safe

Look back at the code you wrote on Days 16, 17, and 18. You will find database passwords hardcoded as strings. This is one of the most common and most consequential mistakes in software development. In 2023, a significant number of data breaches were traced back to secrets committed to source code repositories — including private ones, because people change repository visibility or leave companies.

Today you fix this across your entire codebase. You will also understand the full secret management landscape: why `.env` files exist, why they are not sufficient for production, and what a real production secret management system looks like. This day is short on code and long on understanding — the concept matters more than the implementation.

THEORY · 20 MINUTES

What an environment variable is

An environment variable is a key-value pair that exists in the operating system environment where a process runs. When you start a program, it inherits the environment variables of the process that launched it. This means you can provide configuration to your application without putting it in the source code — the application reads the variable at runtime.

Environment variables are not a security mechanism by themselves. They are simply a way to separate configuration from code. The security benefit is that when configuration is separate from code, you can commit your code to a public repository without committing your secrets. The secrets live in the environment where the code runs, not in the code itself.

The `.env` file and `python-dotenv`

During development, you do not want to set environment variables manually in your terminal every time you start your application. A `.env` file solves this: it is a text file containing key-value pairs that `python-dotenv` loads into the environment when your application starts.

A `.env` file is not secret storage — it is a developer convenience tool. Anyone with access to your file system can read it. The rule is: `.env` goes in `.gitignore` and never gets committed to version control. It lives on your laptop and on the server, not in the repository. If you ever accidentally commit a `.env` file, treat the secrets in it as compromised and rotate them immediately.

The secret management landscape

Term	Meaning
<i>.env file</i>	Developer convenience. Never commit to version control. Sufficient for local development.
<i>Environment variables</i>	The standard runtime mechanism. Set in your deployment environment — Docker, cloud platform, CI/CD pipeline.
<i>Secret managers</i>	AWS Secrets Manager, Google Secret Manager, HashiCorp Vault. Store secrets encrypted, audit access, rotate automatically. The correct solution for production.
<i>Never</i>	Hardcoded strings, config files committed to repos, environment variables logged to console, secrets in error messages.

What to do if you accidentally commit a secret

Act immediately. Treat the secret as compromised the moment it is committed — even if you delete it in the next commit, it still exists in the git history and anyone who cloned the repository before the deletion has a copy.

The steps: rotate the secret first (change the password, revoke the API key), then remove it from git history using git filter-repo or BFG Repo Cleaner, then force-push the cleaned history. Do not simply delete the file and commit — the secret remains in git history and can be recovered with git log.

PRACTICAL · 60 MINUTES

This practical has two phases: audit and clean your existing code, then build a proper configuration system for the rest of the curriculum.

1. Audit your codebase for hardcoded secrets (10 minutes)

Grep your repository for any hardcoded passwords, connection strings, or API keys.

```
# Find hardcoded database passwords in your Python files
grep -rn 'password' week-01/ week-02/ week-03/
grep -rn 'YOUR_PASSWORD' week-01/ week-02/ week-03/
grep -rn 'postgres://' week-01/ week-02/ week-03/

# Check git history for anything sensitive (common mistake)
git log --all --full-history -- '*.env'
git log -p --all | grep -i 'password\|secret\|key' | head -20
```

If you find real secrets in git history, rotate them before continuing

2. Create your .env file and .gitignore entry (10 minutes)

Set up the configuration pattern you will use for the rest of the curriculum.

```
# .env — at the root of your repository
# DO NOT commit this file
DATABASE_URL=postgresql://postgres:your_actual_password@localhost/university_db
SECRET_KEY=change-this-to-a-long-random-string-in-production
DEBUG=true
ENVIRONMENT=development
```

.env — lives in the root of building-backend-systems/, never committed

```
# .gitignore — add these lines
.env
.env.local
.env.*.local
*.env

# Verify .gitignore is working
git status # .env should NOT appear as a tracked or
untracked file
```

.gitignore — verify with git status after adding

3. Build a proper settings module using Pydantic Settings (25 minutes)

Pydantic Settings validates your environment variables at startup — your application fails immediately with a clear error if a required variable is missing, rather than failing with a cryptic error hours later.

```
pip install pydantic-settings
```

Install pydantic-settings

```
# week-03/day19_config.py
from pydantic_settings import BaseSettings,
SettingsConfigDict
from functools import lru_cache

class Settings(BaseSettings):
    model_config = SettingsConfigDict(
        env_file='.env',
        env_file_encoding='utf-8',
        case_sensitive=False,
    )

    # Required — application refuses to start if these are
    # missing
    database_url: str
    secret_key: str

    # Optional with sensible defaults
    environment: str = 'development'
    debug: bool = False
    log_level: str = 'INFO'
```

```

@property
def is_production(self) -> bool:
    return self.environment.lower() == 'production'

# lru_cache ensures Settings is only instantiated once per
# process
@lru_cache
def get_settings() -> Settings:
    return Settings()

# Usage: import and call get_settings() anywhere in the app
# settings = get_settings()
# engine = create_engine(settings.database_url)

```

week-03/day19_config.py — this becomes the foundation for all remaining week files

4. Refactor Day 18 to use Settings (15 minutes)

Update the database configuration to use the Settings module. This is the pattern used in every production FastAPI application.

```

# week-03/day18_database.py — updated version
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker, DeclarativeBase
from .day19_config import get_settings

settings = get_settings()

engine = create_engine(
    settings.database_url,
    echo=settings.debug, # only print SQL in debug mode
)

SessionLocal = sessionmaker(autocommit=False,
                             autoflush=False, bind=engine)

class Base(DeclarativeBase):
    pass

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

```

No hardcoded strings. The application reads from .env at startup and fails immediately if anything is missing.

REFLECTION · 10 MINUTES

Answer these questions — they prepare you for real security decisions.

Question 1

You accidentally committed a database password to your repository three commits ago. What are the exact steps you take? List them in order, starting with the most urgent action.

Question 2

The Settings class uses `@lru_cache` on `get_settings()`. What does this mean in practice? What would happen if you called `get_settings()` on every request without the cache?

Question 3

Your application runs in a Docker container on a cloud platform. The `.env` file is on your laptop but not in the Docker image (it is in `.dockerignore`). How do secrets get into the running container? Name two mechanisms.

Question 4

Pydantic Settings validates your environment variables at startup and raises a `ValidationError` if a required variable is missing. Why is this better than checking for missing variables in the middle of a request handler?

BEFORE TOMORROW

Tomorrow you build authentication — the login system that every application needs. You will implement JWT from scratch, understand what signing means and why the secret key must stay secret, and build a complete login and

protected route flow. The Settings module you built today will be the foundation for storing the JWT secret key.

Commit before you close your laptop

Ensure .env is in .gitignore and NOT tracked by git

Save week-03/day19_config.py

```
git commit -m "Day 19: environment variables, .env, Pydantic Settings,  
secrets management"
```


Day 20

Authentication: Sessions and JWT

Week Three · Making Databases Fast and Safe

Authentication is the process of establishing who someone is. It is different from authorisation, which is the process of establishing what they are allowed to do. Today is authentication. You will build a complete login system from scratch: user registration with hashed passwords, a login endpoint that issues a JWT, and protected routes that verify the token on every request.

You will build this without a library that hides the mechanism. The goal is to understand what a JWT is, why passwords must be hashed and never stored in plain text, why the secret key must stay secret, and why tokens expire. These are not details — they are the concepts that prevent security vulnerabilities.

THEORY · 20 MINUTES

Authentication versus authorisation

Authentication answers the question: who are you? Authorisation answers the question: what are you allowed to do? A locked door with a keypad performs authentication — it checks whether you know the code. The rooms you can access once inside perform authorisation — your badge determines which doors open for you.

This distinction matters in code because they belong in different layers. Authentication is handled at the edge of your application — the middleware or dependency that validates the token before any route handler runs. Authorisation is handled inside your service layer, where business rules about who can do what are enforced. Mixing them produces systems that are hard to test and easy to misconfigure.

Why passwords must be hashed

A hash function takes an input of any length and produces a fixed-length output. The same input always produces the same output. But you cannot reverse the process — you cannot take the output and recover the input. Bcrypt is a hash function designed specifically for passwords: it is intentionally slow (making brute-force attacks expensive), and it includes a random salt (making precomputed rainbow table attacks useless).

You never store a password. You store the hash. When a user logs in, you hash what they provided and compare it to the stored hash. If the hashes match, the password is correct. If your database is ever stolen, the attacker has hashes, not passwords — and bcrypt hashes are expensive enough to crack that most passwords remain safe during the time it takes you to detect the breach and force a password reset.

What a JWT is

A JWT — JSON Web Token — is a string with three parts separated by dots: a header, a payload, and a signature. The header describes the token type and the algorithm used to sign it. The payload contains claims — pieces of information about the user and the token itself, like the user's ID and when the token expires. The signature is a cryptographic hash of the header and payload, created using your secret key.

The signature is what makes JWTs trustworthy. Anyone can read the header and payload — they are just base64-encoded JSON, not encrypted. But only someone with the secret key can create a valid signature. When your server receives a JWT, it verifies the signature using the secret key. If the signature is valid, the server knows the token was created by someone who

had the key — and that the header and payload have not been modified since the token was created.

Access tokens and refresh tokens

An access token is short-lived — typically 15 to 60 minutes. A refresh token is long-lived — typically 7 to 30 days. The access token is sent with every API request. When it expires, the client uses the refresh token to get a new access token. This design limits the damage from a stolen access token: it only works for the duration of its short lifetime. The refresh token is stored more carefully (in an HTTP-only cookie or secure storage) and is only sent to one endpoint — the token refresh endpoint.

PRACTICAL · 60 MINUTES

Today you build the complete authentication system. It will connect to your real PostgreSQL database using the Settings module from Day 19.

```
pip install python-jose[cryptography] passlib[bcrypt]
```

Install JWT and password hashing dependencies

1. Create the users table and password utilities (10 minutes)

Add a users table to `university_db` and build the hashing utilities.

```
-- In psql - add to university_db
CREATE TABLE auth_users (
    id                INTEGER GENERATED ALWAYS AS IDENTITY
    PRIMARY KEY,
    email             TEXT                NOT NULL UNIQUE,
    hashed_password   TEXT                NOT NULL,
    is_active         BOOLEAN            NOT NULL DEFAULT TRUE,
    created_at        TIMESTAMPTZ        NOT NULL DEFAULT NOW()
);
```

Note the column is called `hashed_password` — a reminder to everyone reading the schema

```
# week-03/day20_auth.py
from passlib.context import CryptContext
```

```

from jose import JWTError, jwt
from datetime import datetime, timedelta, timezone
from .day19_config import get_settings

settings = get_settings()
ALGORITHM = 'HS256'
ACCESS_TOKEN_EXP = timedelta(minutes=30)

pwd_context = CryptContext(schemes=['bcrypt'],
deprecatd='auto')

def hash_password(plain: str) -> str:
    return pwd_context.hash(plain)

def verify_password(plain: str, hashed: str) -> bool:
    return pwd_context.verify(plain, hashed)

def create_access_token(user_id: int) -> str:
    payload = {
        'sub': str(user_id),
        'exp': datetime.now(timezone.utc) + ACCESS_TOKEN_EXP,
        'iat': datetime.now(timezone.utc),
    }
    return jwt.encode(payload, settings.secret_key,
algorithm=ALGORITHM)

def decode_access_token(token: str) -> int | None:
    """Returns the user_id if the token is valid, None
otherwise."""
    try:
        payload = jwt.decode(token, settings.secret_key,
algorithms=[ALGORITHM])
        return int(payload['sub'])
    except JWTError:
        return None

```

week-03/day20_auth.py

2. Build the registration and login endpoints (25 minutes)

Build the complete auth flow in FastAPI. The protected route dependency is the most important part — study it carefully.

```

# week-03/day20_routes.py
import psycpg2
from fastapi import FastAPI, HTTPException, Depends, status

```

```

from fastapi.security import OAuth2PasswordBearer,
OAuth2PasswordRequestForm
from pydantic import BaseModel
from .day20_auth import hash_password, verify_password,
create_access_token, decode_access_token
from .day19_config import get_settings

app = FastAPI()
settings = get_settings()
DB = settings.database_url
oauth2 = OAuth2PasswordBearer(tokenUrl='/auth/login')

class RegisterIn(BaseModel):
    email: str; password: str

# — FastAPI dependency that validates the token on every
# protected request —
def get_current_user(token: str = Depends(oauth2)) -> dict:
    user_id = decode_access_token(token)
    if user_id is None:
        raise HTTPException(status.HTTP_401_UNAUTHORIZED,
            detail='Invalid or expired token',
            headers={'WWW-Authenticate':
                'Bearer'})
    conn = psycopg2.connect(DB)
    cur = conn.cursor()
    cur.execute('SELECT id, email FROM auth_users WHERE id =
%s AND is_active = TRUE', (user_id,))
    row = cur.fetchone(); conn.close()
    if not row:
        raise HTTPException(status.HTTP_401_UNAUTHORIZED,
            detail='User not found')
    return {'id': row[0], 'email': row[1]}

@app.post('/auth/register', status_code=201)
def register(data: RegisterIn):
    conn = psycopg2.connect(DB)
    cur = conn.cursor()
    try:
        cur.execute('INSERT INTO auth_users (email,
            hashed_password) VALUES (%s, %s) RETURNING id',
            (data.email,
            hash_password(data.password)))
        conn.commit()
        return {'id': cur.fetchone()[0], 'email': data.email}

```

```

except psycopg2.errors.UniqueViolation:
    conn.rollback()
    raise HTTPException(409, 'Email already registered')
finally:
    conn.close()

@app.post('/auth/login')
def login(form: OAuth2PasswordRequestForm = Depends()):
    conn = psycopg2.connect(DB)
    cur = conn.cursor()
    cur.execute('SELECT id, hashed_password FROM auth_users
WHERE email = %s', (form.username,))
    row = cur.fetchone(); conn.close()
    if not row or not verify_password(form.password, row[1]):
        raise HTTPException(401, 'Incorrect email or
password')
    return {'access_token': create_access_token(row[0]),
'token_type': 'bearer'}

@app.get('/me')
def me(user: dict = Depends(get_current_user)):
    return user # protected route - only reachable with a
valid token

```

week-03/day20_routes.py — test the full flow in Postman: register → login → call /me with the token

3. Test expiry by manipulating the token lifetime (25 minutes)

Set ACCESS_TOKEN_EXP to 10 seconds, register and log in, wait 15 seconds, then call /me. Observe the 401. Then implement token refresh.

```

# In day20_auth.py - temporarily change to 10 seconds for
testing
ACCESS_TOKEN_EXP = timedelta(seconds=10)

# Test flow in Postman:
# 1. POST /auth/register - create a user
# 2. POST /auth/login - get a token
# 3. GET /me - succeeds immediately
# 4. Wait 15 seconds
# 5. GET /me - returns 401 Unauthorized

# Then: decode the expired token manually to see the exp
claim

```

```
# (JWTs are base64-encoded — paste the token into jwt.io in  
your browser)
```

jwt.io decodes any JWT without verifying the signature — useful for debugging

REFLECTION · 10 MINUTES

These questions address the security decisions embedded in today's code.

Question 1

The login endpoint returns the same error message — 'Incorrect email or password' — whether the email does not exist or the password is wrong. Why is this deliberate? What information would an attacker gain if you returned different messages for each case?

Question 2

The JWT payload contains the user ID and expiry time. It does not contain the password or any sensitive information. If someone steals a JWT, what can they do? What can they not do? What limits the damage?

Question 3

You store the JWT secret key in the .env file and load it through Pydantic Settings. What happens to authentication for all current users if you change the secret key? When would you intentionally do this?

Question 4

The `get_current_user` dependency queries the database on every protected request to verify the user still exists and is still active. This adds a database query to every API call. What is the tradeoff between this approach and trusting the JWT without a database lookup?

BEFORE TOMORROW

Tomorrow is the Week Three summary day. Before the summary, read through all the code you wrote this week from Day 15 through today. Notice how each day built directly on the previous one — indexes made queries fast, transactions made writes safe, locks handled concurrency, the ORM abstracted the database, settings cleaned up the configuration, and authentication secured the API. These six concepts together form the foundation of a production-ready backend.

Commit before you close your laptop

Save all week-03/day20_*.py files

```
git commit -m "Day 20: JWT authentication, password hashing, protected routes"
```


Week 3 Summary

Consolidation and Reflection

No new concept today. Week Three was the most technically dense week of the curriculum so far — six distinct concepts across six days, each one building on the previous. The goal today is to consolidate them into a coherent mental model, identify any gaps, and push a clean and well-organised repository.

The diagnostic for this week is not a single query. It is a design question that requires you to apply everything simultaneously: if you were building a financial transaction endpoint for Pivota — an endpoint that records a MoMo payment and updates a balance — what would you need to apply from Week Three, and why?

THE WEEKLY SUMMARY

Write five to ten sentences answering this question: you are building an endpoint for Pivota that receives a MoMo payment webhook, records the transaction, and updates the user's balance. Walk through every concern from Week Three that applies to this endpoint and explain what you would do about each one.

The concerns to address: how would you make the balance update query fast at scale? Why does the entire operation need to be in a transaction? What concurrency problem could occur and how would you prevent it? Where would the database credentials live? How would you verify the request is coming from an authenticated source?

Write this before looking at your notes

If you can answer this question completely from memory, Week Three is solid.

If a specific concern draws a blank, go back to that day's material before Week Four.

Save your answer in `week-03/SUMMARY.md`

REVIEW CHECKLIST

Work through each item. These are the specific skills Week Three aimed to build.

1. Can you create an index in PostgreSQL and use `EXPLAIN ANALYZE` to verify it is being used?
2. Can you explain what a B-Tree index is and why it makes lookups $O(\log n)$ instead of $O(n)$?
3. Can you explain ACID in plain English using a concrete real-world example that is not a bank transfer?
4. Can you write a Python function that performs a multi-step database operation atomically, rolls back on failure, and returns a clear error?
5. Can you explain what a race condition is and reproduce the counter experiment from memory?
6. Can you explain the difference between atomic SQL updates, pessimistic locking, and optimistic locking — and give a concrete use case for each?
7. Can you set up a Pydantic Settings module that validates environment variables and fails fast on missing values?
8. Can you explain why passwords are hashed rather than encrypted, and what makes `bcrypt` specifically suitable for passwords?
9. Can you explain what a JWT signature proves and what it does not prove?

THE CONNECTING THREAD

Week Three introduced six concepts that appear to be independent but are deeply connected. Reading them in order reveals a pattern.

Indexes solve the problem of finding data efficiently in large tables. But fast reads are only useful if the data is correct — which brings you to transactions. Transactions ensure that multi-step operations are atomic — but they only protect against sequential failures, not concurrent ones. Concurrency control protects against simultaneous operations corrupting data — but only if you reach the database at all. Getting to the database requires credentials, which must be managed securely — secrets management. And finally, knowing who is making requests requires authentication.

These six concepts form a layer cake from the storage layer up to the API layer. Each one is necessary but not sufficient on its own. An application with fast indexes but no transactions is fast and incorrect. An application with transactions but no concurrency control is correct in single-threaded tests and wrong in production. An application with everything else but hardcoded secrets is a breach waiting to happen.

CONSOLIDATION EXERCISE

Write the following in a new file called `week-03/day21_integration.py`. Do not look at notes — write from memory, looking things up only if you are genuinely stuck on syntax, not on concepts.

Build a protected transfer endpoint

A FastAPI endpoint `POST /transfer` that requires a valid JWT to call.

The request body contains: `from_account_id`, `to_account_id`, `amount`.

The endpoint must: verify the JWT, check the sender owns the from account, perform the transfer atomically using `SELECT FOR UPDATE`, and return the new balances.

If the balance is insufficient, return 400 with a clear message.

If either account is not found, return 404.

No hardcoded credentials anywhere.

This exercise deliberately combines authentication (Day 20), transactions (Day 16), and concurrency control (Day 17). You do not need to be perfect — the goal is to identify which parts feel natural and which require looking up. The parts you look up are the parts worth reviewing before Week Four.

FINAL COMMIT

Before you close your laptop, your repository should contain:

10. week-03/day15_seed.py — the 500,000-row seeding script
11. week-03/day15_indexes.sql — EXPLAIN ANALYZE queries and index creation
12. week-03/day16_transactions.sql — accounts schema and constraint tests
13. week-03/day16_transfer.py — the atomic transfer function
14. week-03/day17_broken_counter.py — the race condition demonstration
15. week-03/day17_hammer.py — the concurrent request test
16. week-03/day17_fixed_counter.py — three concurrency fixes
17. week-03/day18_database.py — SQLAlchemy engine and session
18. week-03/day18_models.py — Student, Course, Enrollment models
19. week-03/day18_n1_demo.py — N+1 problem and joinedload fix
20. week-03/day19_config.py — Pydantic Settings module
21. week-03/day20_auth.py — password hashing and JWT utilities
22. week-03/day20_routes.py — register, login, and protected route
23. week-03/day21_integration.py — the consolidation exercise
24. week-03/SUMMARY.md — your written weekly summary

```
git add .  
git commit -m "Week 3 complete: indexes, transactions,  
concurrency, ORM, secrets, auth"  
git push
```

Your final commit for Week Three

WEEK FOUR PREVIEW

Week Four is production-grade backend engineering. You have a database, an ORM, authentication, and clean configuration. Now you make the application behave correctly and safely under real conditions: authorisation with roles and permissions, caching expensive queries with Redis, global error handling, design patterns that make your codebase maintainable, structured logging, API security, and background jobs for operations that should not block a request.

The shift in Week Four is from correctness to robustness. Your application is technically correct at the end of Week Three. Week Four makes it the kind of application that survives contact with real users, real load, and real failure scenarios. The concepts this week feel different from the database-heavy content of Weeks Two and Three — but they are what separates a backend that works in development from one that works in production.